

Deep Galerkin Method (DGM)

A Deep Learning Algorithm for Solving PDEs

Based on Sirignano & Spiliopoulos, *J. Comput. Phys.*, 2018

Aryan Deo

April 14, 2026

Outline

- 1 Motivation and Problem Statement
- 2 The DGM Algorithm
- 3 DGM Network Architecture
- 4 Convergence Theory
- 5 Numerical Results
- 6 Paper's High-Dimensional Results
- 7 Conclusion

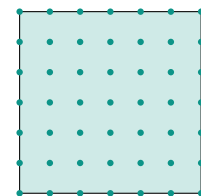
The Curse of Dimensionality in PDEs

Traditional methods fail in high dimensions:

- Finite differences need $O(N^{d+1})$ grid points
- $d = 3, N = 100$: 10^8 points (feasible)
- $d = 100, N = 100$: 10^{200} points (impossible!)

High-dim PDEs arise in:

- Finance: option pricing on portfolios
- Physics: many-body quantum systems
- Engineering: stochastic control
- Biology: population dynamics



2D mesh: 49 pts $d \uparrow$



N^{d+1}

INFEASIBLE

Key Idea: Replace the mesh with a neural network trained on random points!

Core Idea: PDE as an Optimization Problem

Consider a parabolic PDE:

$$\frac{\partial u}{\partial t}(t, x) + \mathcal{L}u(t, x) = 0, \quad u(0, x) = u_0(x), \quad u|_{\partial\Omega} = g$$

Step 1: Approximate $u(t, x)$ with a neural network $f(t, x; \theta)$

Step 2: Define a loss measuring how well f satisfies the PDE:

$$J(\theta) = \underbrace{\left\| \frac{\partial f}{\partial t} + \mathcal{L}f \right\|^2}_{\text{PDE residual}} + \lambda_1 \underbrace{\|f - g\|_{\partial\Omega}^2}_{\text{BC error}} + \lambda_2 \underbrace{\|f(0, \cdot) - u_0\|^2}_{\text{IC error}}$$

Step 3: Minimize $J(\theta)$ via SGD on **randomly sampled** points

If $J = 0$, then f is an exact solution!

No knowledge of the true solution is needed.

DGM Algorithm: Step by Step

Algorithm 1 Deep Galerkin Method

- 1: **Initialize** network parameters θ_0
 - 2: **repeat**
 - 3: Sample interior points $(t_n, x_n) \sim \nu_1$ on $[0, T] \times \Omega$
 - 4: Sample boundary points $(\tau_n, z_n) \sim \nu_2$ on $[0, T] \times \partial\Omega$
 - 5: Sample initial points $w_n \sim \nu_3$ on Ω
 - 6: Compute loss $G(\theta_n, s_n)$ using **automatic differentiation**
 - 7: Update: $\theta_{n+1} = \theta_n - \alpha_n \nabla_{\theta} G(\theta_n, s_n)$
 - 8: **until** convergence
-

Key properties:

- $\nabla_{\theta} G$ is an **unbiased** estimate of $\nabla_{\theta} J$
- No mesh ever constructed — **meshfree**
- Points processed sequentially — memory efficient
- Implemented with PyTorch/TensorFlow automatic differentiation

LSTM-Inspired Architecture (Eq. 4.2 in paper)

Standard MLPs struggle with sharp gradients. The DGM network uses **gating**:

$$S^1 = \sigma(W^1 \vec{x} + b^1) \quad (\text{initial layer})$$

$$Z^\ell = \sigma(U^z \vec{x} + W^z S^\ell + b^z) \quad (\text{update gate})$$

$$G^\ell = \sigma(U^g \vec{x} + W^g S^\ell + b^g) \quad (\text{gate})$$

$$R^\ell = \sigma(U^r \vec{x} + W^r S^\ell + b^r) \quad (\text{reset gate})$$

$$H^\ell = \sigma(U^h \vec{x} + W^h (S^\ell \odot R^\ell) + b^h) \quad (\text{candidate})$$

$$S^{\ell+1} = (1 - G^\ell) \odot H^\ell + Z^\ell \odot S^\ell \quad (\text{state update})$$

$$f(t, x; \theta) = W S^{L+1} + b \quad (\text{output})$$

Hyperparameters used: $L = 3$ layers, $M = 50$ units, $\sigma = \tanh$

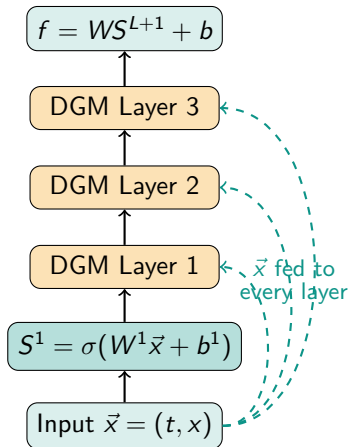
Why Gating Helps

Standard MLP:

- Fixed transformations per layer
- Struggles with sharp transitions
- Vanishing gradient in deep networks

DGM Network:

- Gates control information flow
- Input $\vec{x}$ injected at **every** layer
- Element-wise multiplication creates complex nonlinear interactions
- Analogous to LSTM/Highway networks



Theoretical Guarantees

Theorem (Sirignano & Spiliopoulos, 2018)

For a class of quasilinear parabolic PDEs with smooth coefficients, let f^n be neural networks minimizing $J(f)$. Then:

- 1 $J(f^n) \rightarrow 0$ as $n \rightarrow \infty$ (approximation power)
- 2 $f^n \rightarrow u$ strongly in $L^\rho(\Omega_T)$ for $\rho < 2$ (convergence to PDE solution)

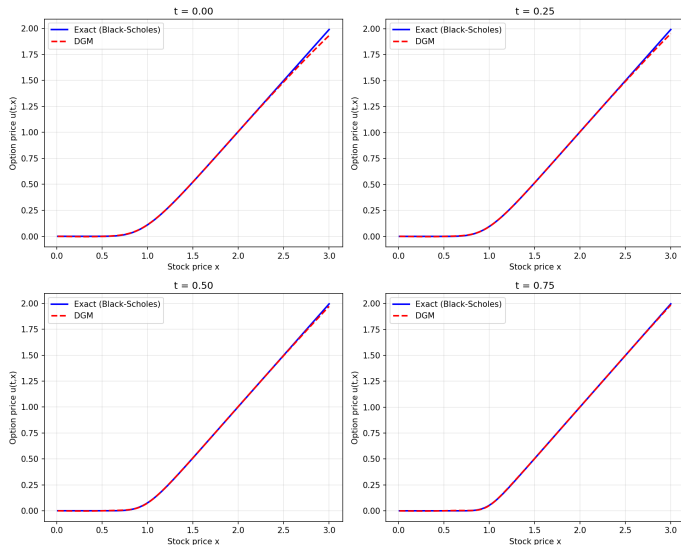
Proof sketch:

- 1 Universal approximation (Hornik, 1991) $\Rightarrow J(f^n) \rightarrow 0$
- 2 Each f^n satisfies a PDE with vanishing source h^n
- 3 Energy estimates $\Rightarrow$ uniform bounds in $L^\infty(0, T; L^2(\Omega))$
- 4 Aubin-Lions compactness $\Rightarrow$ subsequential convergence
- 5 Vitali's theorem $\Rightarrow$ pass to limit in weak formulation

Significance: First rigorous convergence proof for neural network PDE solvers.

Result 1: Black-Scholes PDE

DGM Solution vs Exact Black-Scholes



PDE:

$$\frac{\partial u}{\partial t} + \frac{\sigma^2 x^2}{2} \frac{\partial^2 u}{\partial x^2} + (r - c)x \frac{\partial u}{\partial x} - ru = 0$$

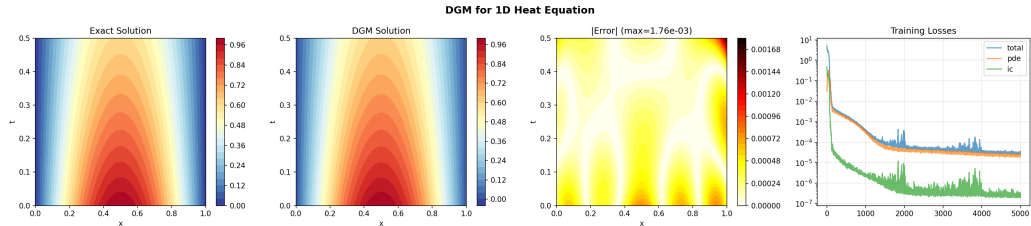
Parameters:

$$\sigma = 0.25, r = 0.05$$

$$c = 0.02, K = 1, T = 1$$

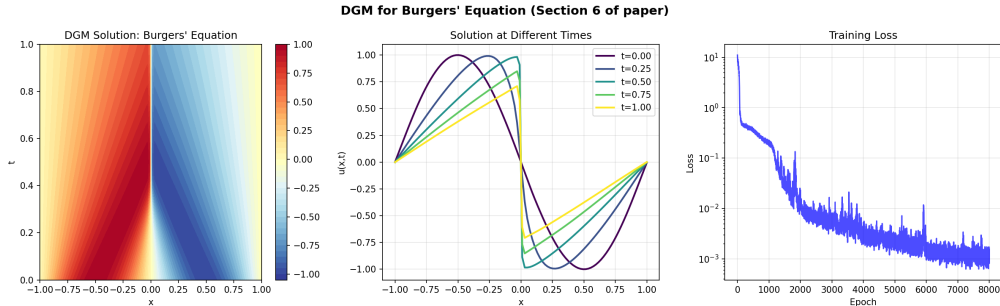
DGM captures the option price profile including the kink near the strike.

Result 2: Heat Equation



$$\frac{\partial u}{\partial t} = 0.1 \frac{\partial^2 u}{\partial x^2}, \quad u(x, 0) = \sin(\pi x), \quad u(0, t) = u(1, t) = 0$$

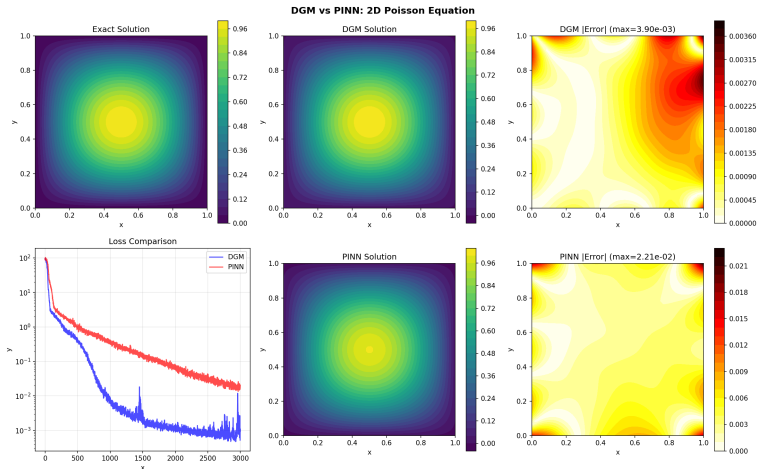
Result 3: Burgers' Equation



$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \frac{0.01}{\pi} \frac{\partial^2 u}{\partial x^2}, \quad u(x, 0) = -\sin(\pi x)$$

DGM captures the shock formation as the solution steepens over time.

Result 4: 2D Poisson Equation



$$\nabla^2 u = -2\pi^2 \sin(\pi x) \sin(\pi y), \quad u|_{\partial\Omega} = 0$$

DGM successfully solves the elliptic PDE (not just parabolic!).

High-Dimensional Results from the Paper

Problem	Dimensions	Error
American Option (Black-Scholes)	3	0.05%
American Option (Black-Scholes)	20	0.03%
American Option (Black-Scholes)	100	0.11%
American Option (Black-Scholes)	200	0.22%
Hamilton-Jacobi-Bellman	21	0.1% avg
Burgers' (parameterized)	1+4 params	Visually accurate

Table: Results from Sirignano & Spiliopoulos (2018), Tables 1–2

200-dimensional PDE solved with 0.22% error!

Trained on Blue Waters supercomputer with 6 GPU nodes, $\sim 500\text{M}$ sample points

Summary of implementation results

Problem	Domain	Epochs	Typical Error
Black-Scholes PDE	$[0, T] \times [0, 3]$	5000	$\sim 1.25\%$ relative
Heat Equation	$[0, 1] \times [0, 1]$	5000	1.76×10^{-3}
Burgers' Equation	$[-1, 1] \times [0, 1]$	8000	$\sim 10^{-3}$
2D Poisson (DGM)	$[0, 1]^2$	3000	3.90×10^{-3}

Summary and Takeaways

What DGM offers:

- Meshfree — no curse of dimensionality
- Continuous, differentiable solution
- Rigorous convergence theory
- LSTM-inspired architecture for sharp gradients
- Handles parabolic, elliptic, free boundary PDEs

Open challenges:

- Computationally expensive training
- Non-convex optimization (local minima)
- Oscillatory solutions remain difficult
- Architecture selection is problem-dependent
- Error estimation during training

Impact

DGM opened the door to solving PDEs in hundreds of dimensions, with applications spanning finance (option pricing), control theory (HJB equations), and fluid dynamics (Navier-Stokes). It has inspired the broader PINNs/SciML movement.

References

- ① J. Sirignano, K. Spiliopoulos, “DGM: A deep learning algorithm for solving PDEs,” *J. Comput. Phys.*, 375, 2018.
- ② M. Raissi et al., “Physics-informed neural networks,” *J. Comput. Phys.*, 378, 2019.
- ③ I. E. Lagaris et al., “Artificial neural networks for solving ODEs and PDEs,” *IEEE Trans. Neural Netw.*, 9(5), 1998.
- ④ K. Hornik, “Approximation capabilities of multilayer feedforward networks,” *Neural Networks*, 4, 1991.
- ⑤ S. Hochreiter, J. Schmidhuber, “Long short-term memory,” *Neural Computation*, 9(8), 1997.
- ⑥ W. E, J. Han, A. Jentzen, “Deep learning-based numerical methods for high-dimensional parabolic PDEs and BSDEs,” 2017.

Thank You

Questions & Discussion